

Lecture 3: Data Types, Data Collection

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- Thanks for filling out the intro survey
- Office hour schedule:
 - Anna on Zoom: Tue 11 am-12 pm
 - TAs in person: Tue 12-1 pm, Fri 9-10 am, Fri 4-5 pm
 - TAs on Zoom: Thu 11 am-12pm
- In-person office hour location: IRB 2216
- Zoom office hour location: Zoom tab on Canvas
- Will be removing lecture 1 and 2 videos after this week
- Quiz grades will be stored on Gradescope and ported to Canvas

This week we will mostly focus on

- Data Representations and Formats
- Data Collection and Organization
- Experimental Design (Wednesday)



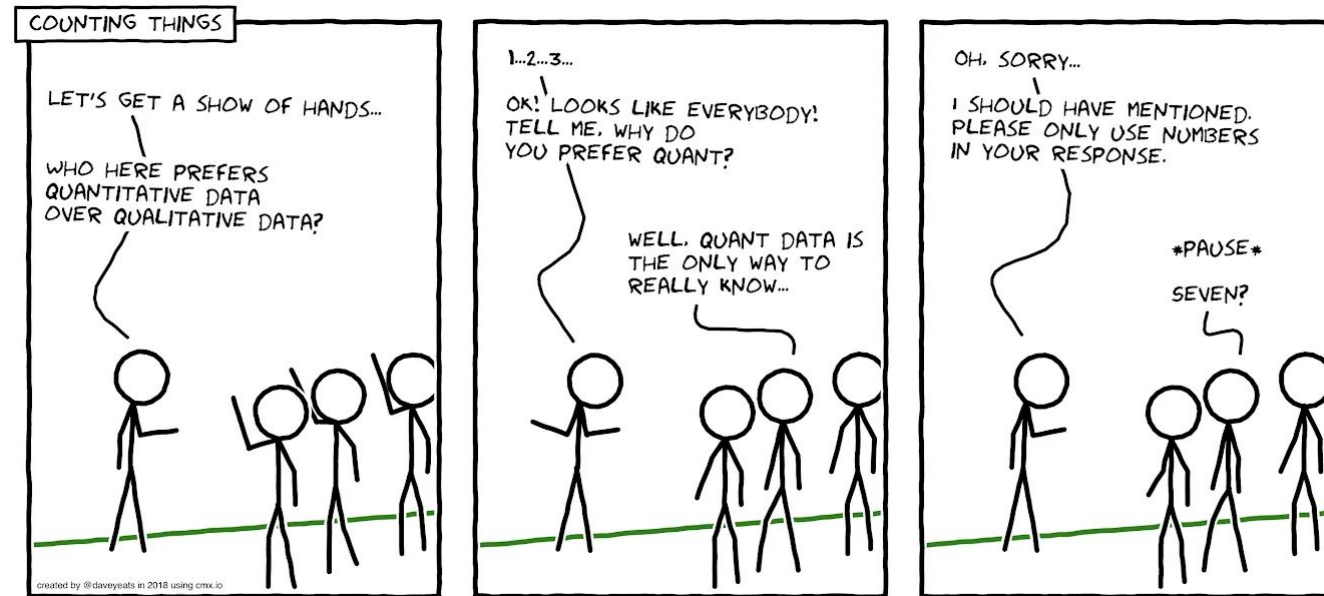
Categories of Data

Quantitative

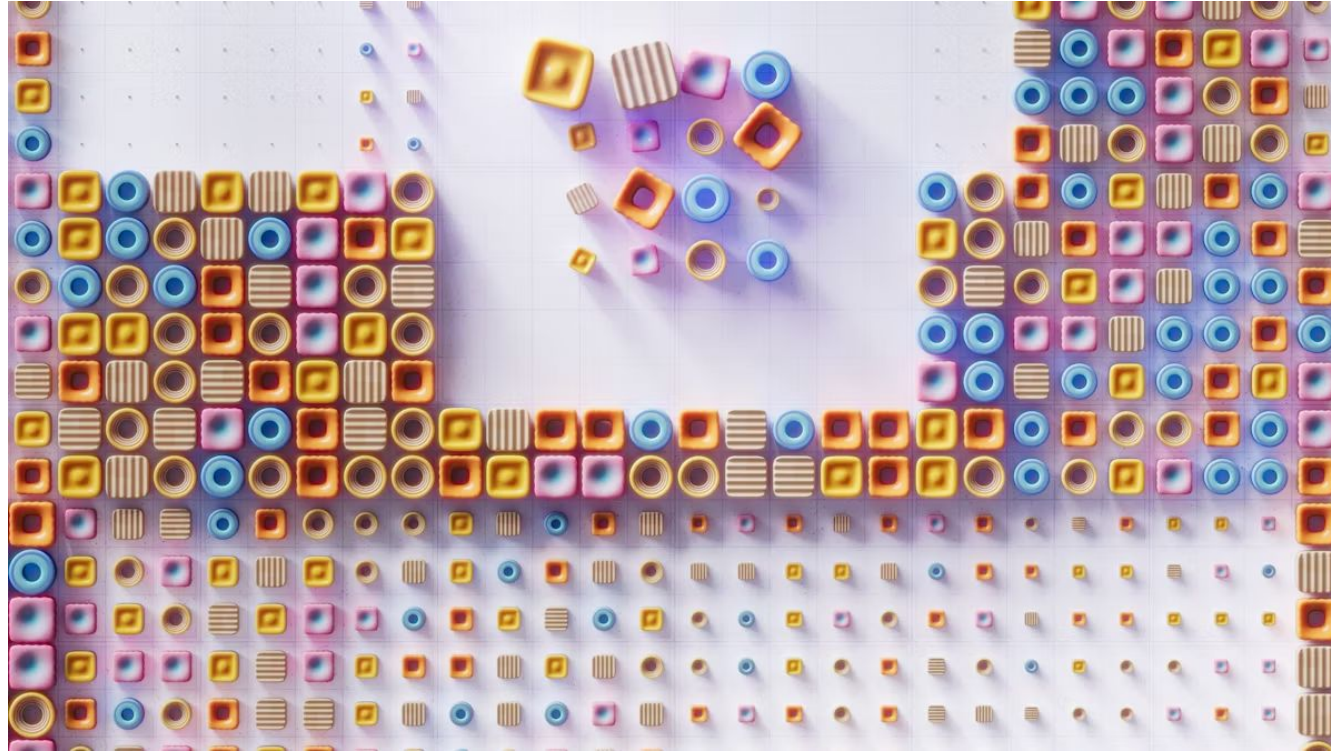
- Numerical
- Measurable
- Ex.
 - How much?
 - How many?
- Types
 - Discrete
 - Continuous

Qualitative

- Non-numerical
- Descriptive / interpretive
- Ex.
 - Why?
 - How?
 - Things we sense: how something feels, tastes, smells



Data Formats



Examples

- **Delimited**

- .csv
- .tsv

- **Image**

- .jpg
- .png

- **Audio**

- .wav
- .mpg

- **Databases**

- **Relational / SQL**
 - MySQL
 - Postgres
- **Non-Relational / NoSQL**
 - Mongo
 - Accumulo

- **Name-Value pairs**

- JSON

- **Tree-based tags**

- XML
- HTML

Data Format: Delimited Files

- **CSV** (**C**omma-**S**eparated **V**alues)
 - A plain-text format where data values are separated by **commas**
- **TSV** (**T**ab-**S**eparated **V**alues)
 - A plain-text format where data values are separated by **tabs**
- Both are common file formats used to store structured data in a simple **tabular format**, with rows and columns.
 - Easy to import and export across various data analysis tools and programming languages.

Delimiter: The separator character: comma (,), tab (\t), colon (:), semicolon (;) etc.

Tabular Data: Example

Detail Compact Column								10 of 13 columns ▾	
▲ Artist	▲ Music	▲ Album	# Year	▲ Genre	# 2022	# 2021			
The Black Crowes	Remedy	The Southern Harmony and Musical Companion	1992	Southern Rock	500				
Asia	Only Time Will Tell	Asia	1982	Progressive Rock	499				
Collective Soul	Shine	Hints Allegations and Things Left Unsaid	1993	Alternative Rock	498				
Billy Idol	Sweet Sixteen	Whiplash Smile	1986	Rock	497				
Collective Soul	December	Collective Soul	1995	Alternative Rock	496				
Duran Duran	Save a prayer	Rio	1982	Synthpop	495	466			
Men at Work	Down Under	Business as Usual	1981	New Wave	494				
Brian Setzer	Summertime Blues	La Bamba soundtrack	1987	Rock and Roll	493				
Simple Minds	Dont You Forget About Me	The Breakfast Club Original Motion Picture Soundtrack	1985	Pop Rock	492				

Representation of tabular data:

Example of how data can be structured in a CSV

```
Artist,Music,Album,Year,Genre,2022,2021,2020,2019,2018,2017,2016,2015
```

```
The Black Crowes,Remedy,The Southern Harmony and Musical  
Companion,1992,Southern Rock,500,,324,290,132,64,36,
```

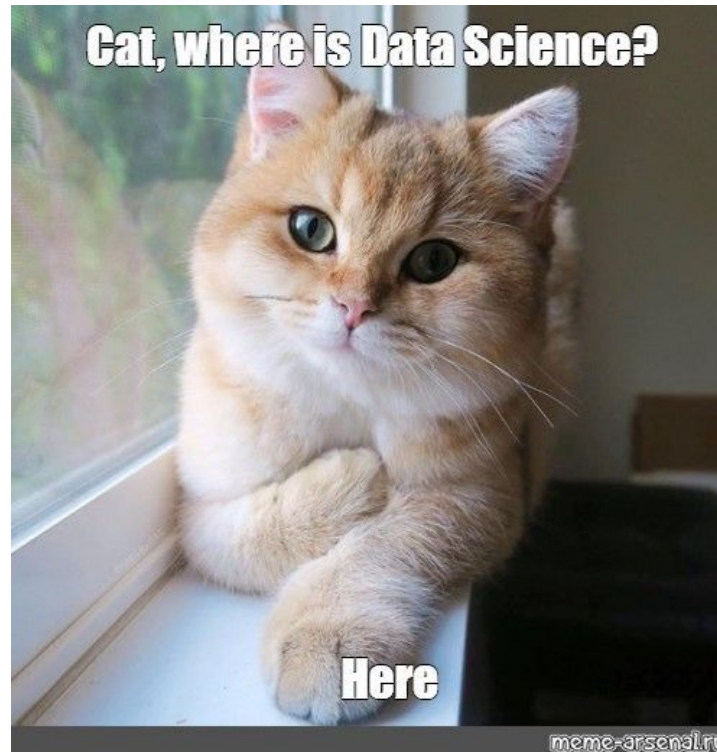
```
Asia,Only Time Will Tell,Asia,1982,Progressive Rock,499,,,,,,,,
```

```
Collective Soul,Shine,Hints Allegations and Things Left Unsaid,1993,Alternative  
Rock,498,,,419,485,403,,
```

- CSV (Comma-Separated Values) files are a common way to store tabular data.
- Python's pandas library makes it easy to load (`df = pd.read_csv('data.csv')`) and manipulate CSV data (*more on this later*)

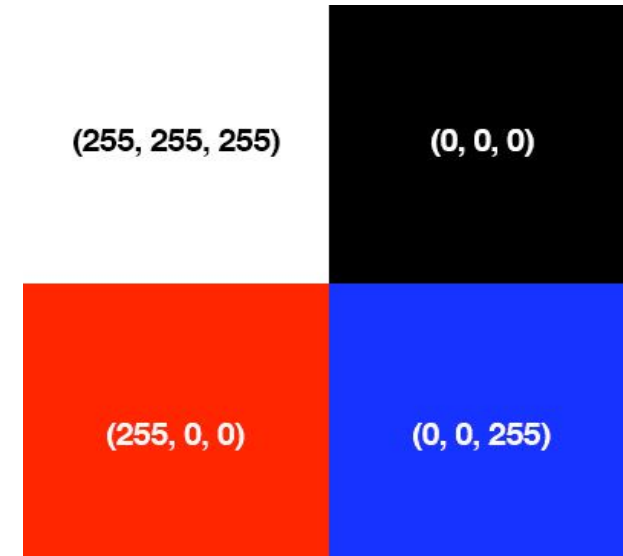
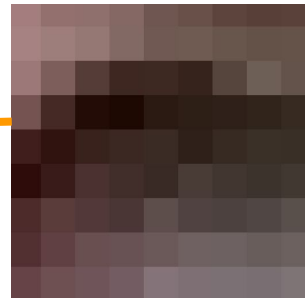
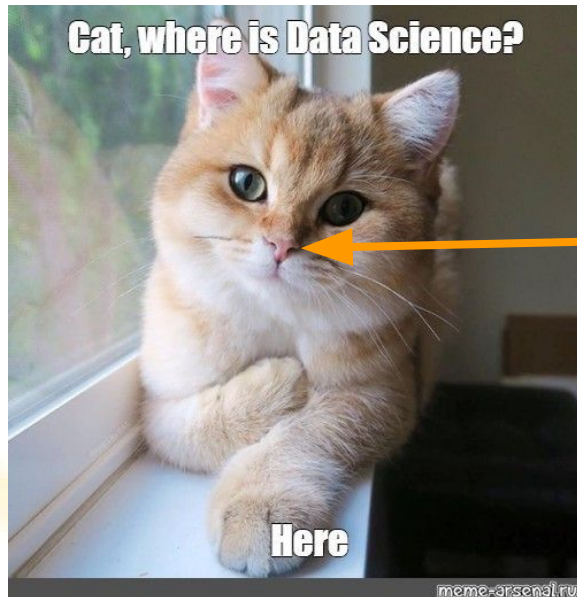
Data Format: Images

Image data encompasses the visual content and properties of an **image** (visual representation), including details such as **colors**, **shapes**, **patterns**, etc.



Data Format: Images

- **Pixel Structure:** Images are organized as grids of **pixels** (tiny building block - square of color).
- **Pixel Data:** Each pixel is assigned a color value, typically using the RGB (**red**, **green**, and **blue** “channels”) color model
- **Values:** Each channel ranges from 0 (no intensity) to 255 (maximum intensity).
- **Color Depth:** The number of possible color values per pixel (ex. 8-bit for 256 colors, 24-bit for millions of colors)
- **Resolution:** width and height of the image, in pixels
- **Binary encoding:** Pixel values are converted into binary for compact file storage



JSON

- JSON stands for **J**ava**S**cript **O**bject **N**otation
- A lightweight data interchange format
- JSON is "self-describing" and easy to understand
 - Easy for humans to read and write
 - Easy for machines to parse and generate
 - Popular for representing data in APIs, configuration files, etc

Example:

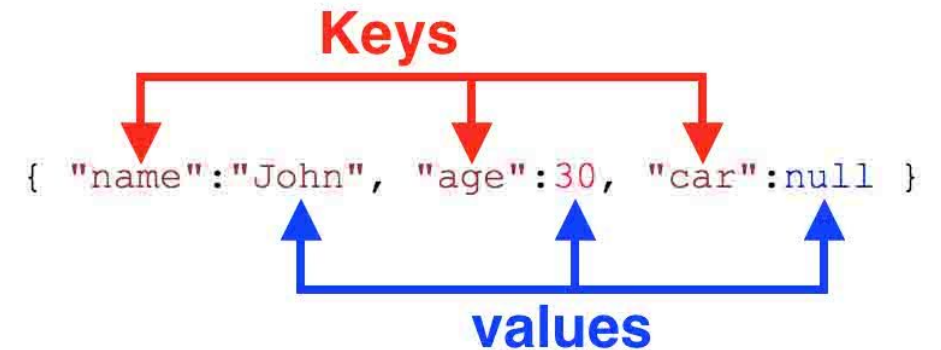
```
'{"name":"John", "age":30, "car":null}'
```

JSON Structure

- Represents data as **key-value pairs** organized in a hierarchical structure using objects and arrays
 - Objects (collections of key-value pairs)
 - Arrays (ordered lists of values)
- Accommodates various data types
 - strings, numbers, arrays, objects, booleans, null
- Allows for nested structures

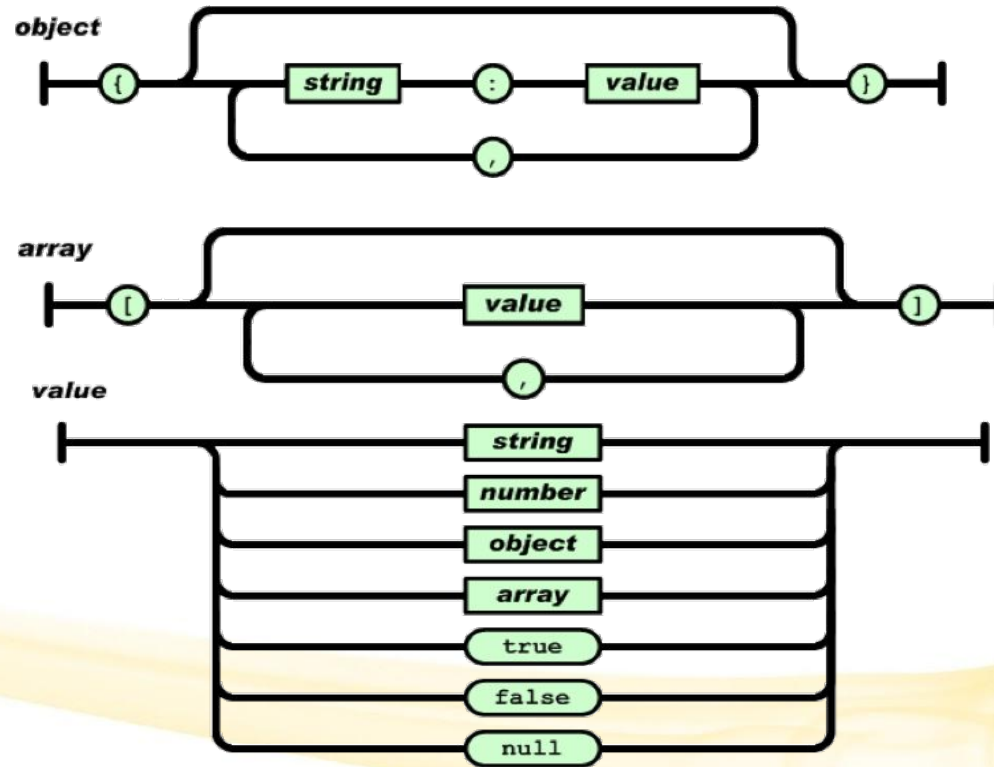
Example:

```
'{"name":"John", "age":30, "car":null}'
```



JSON Files & Strings

- Popular for data interchange and storage
 - Simple and broad compatibility
 - **Serialization** converts an object into a string
 - **Deserialization** converts a string back to an object
- Defined by three universal data structures
 - Object
 - Array
 - Value



```
{
  "string val": "UMD is the place to be",
  "numeric val": 97,
  "boolean val": true,
  "null val": null,
  "object val": {
    "obj_field_1": 42,
    "obj_field_2": "hello"
  },
  "array val1": [
    3,
    4,
    5,
    6
  ],
  "array val2": [
    {
      "id": 0,
      "color": "yellow"
    },
    {
      "id": 1,
      "color": "red"
    }
  ],
  "array val3": [
    "hi",
    "how",
    "are",
    "you"
  ]
}
```


JSON In Python – Understanding Types

- Built-in types
 - ex. `"strings", 1.0, True, False, None`
- Lists
 - ex. `["Nice", "to", "Meet", "You"]`
- Dictionaries
 - ex. `{"hello": "bonjour", "goodbye": "au revoir"}`
- Nested Structures
 - ex. Dictionaries within lists within dictionaries within lists
 - ex. `[1, 2, {"Yay": [ "I", {"am": "in"}, "320" ]}]`

XML / HTML

- HTML = **H**ypertext **M**arkup **L**anguage
- XML = e**X**tensible **M**arkup **L**anguage

Both are common *markup languages* (define the text document within tag, which defines the structure).

Both used for structuring and organizing content, but they serve *different purposes*.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <title>Example</title>
5          <link rel="stylesheet" href="st;
6      </head>
7      <body>
8          <h1>
9              <a href="/">Header</a>
10         </h1>
11         <nav>
12             <a href="one/">One</a>
13             <a href="two/">Two</a>
14             <a href="three/">Three</a>
15         </nav>
```

HTML

```
<person>
  <address>
    <first_name>Peter</first_name>
    <last_name>Miller</last_name>
    <street>Hauptstrasse</street>
    <number>20</number>
    <city>Zurich</city>
  </address>
</person>
```

XML

HTML

- HTML is primarily used for representing **web pages** and *presenting* content on the internet.
- Focused mostly on a fixed set of predefined tags for structuring content, emphasizing elements like headings, paragraphs, links, etc.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <title>Example</title>
5          <link rel="stylesheet" href="st
6      </head>
7      <body>
8          <h1>
9              <a href="/">Header</a>
10         </h1>
11         <nav>
12             <a href="one/">One</a>
13             <a href="two/">Two</a>
14             <a href="three/">Three</a>
15         </nav>
```

Data Organization

Data Organization

The process of organizing data in a logical manner such that it is easier to read and analyze.

Considerations

- Consistency
- Naming
- Data labeling
- Folder structure
- File structure
- Metadata
 - ex. Frequency tables
- Redundancy

*Thinking about this early
can help you collect data
more effectively!*

Data Collection

Recall that Data Collection is...

...a systematic approach to gather **relevant information** from a variety of **sources**.

It is a systemic process of collecting information about a population you wish to study.



How to Get Data?

- Given to you by your company
 - Gathered from external databases
 - From the internet
 - From an API
- 

What is an API?

Requests library and Restful APIs

RESTful APIs (Representational State Transfer Application Programming Interfaces) provide a **structured and documented way** to access data from websites or services.

The **requests** python library allows you to handle HTTP requests/responses very easily

```
▶ import requests  
  response = requests.get("http://api.open-notify.org/astros.json")  
  print(response)
```

```
↔ <Response [200]>
```

APIs: "If you send me a specific request, I will return some information in a structured and documented format."

Why does this matter?

- Data Types
- File Formats
- Databases
- Data Acquisition

Understanding these elements is crucial for progressing through the data science lifecycle, leading to data-driven insights and solutions.

